



## **PROBLEM SESSION 7: Concurrency and Thread**

### **Objective**

The objective of this problem session is to gain experience in writing multi-threaded programs and using explain how race condition effects a shared variable.

### **Instructions**

Write a multi-threaded program that has a race condition. **Think of your own example** which is different from the example given in the lecture.

Compile and run the program to show that its behavior is not deterministic, i.e., each time you run the program, the results may be different. You may need to force an untimely context switch between threads by calling the yield or sleep function – see example below.

### **Untimely Context Switch**

1. Write a multi-threaded program where multiple threads increase/decrease the same variable concurrently without proper synchronization.

**Hint:** To compile and run the program, use the following commands:

```
gcc -o ps7 ps7.c -lpthread  
./ps7
```

2. Run your program **with only one thread** to make sure that your program runs correctly and save results as task 1.
3. Modify your program to create more than one thread.
4. Run your program **without forcing context switching** and save as task 2. You may need to run this task several times to see non-deterministic results.

E.g.

```
counter = counter - 1;
```

5. Implement a program that **force a context switching** while updating a variable.

E.g.

```
register reg = counter; // load memory to register  
reg = reg + 1; // increase register  
yield(); // force context switch  
counter = reg; // save register back to memory
```

6. run the program multiple times and save results as task 3 to show that its behavior is non deterministic. The final value of the shared counter should differ across runs due to the race condition.
7. Discuss what happens and the cause of the race condition in your program.

### **Submission**

Submit **the program code** and **a report file** of examples of your output using LEB2. This should include the code with the race condition, and a short report (in pdf format) giving some explanation of the code and showing the results of running.

Students should work in their existing groups with up to **3-4** students.